

DELULU: A Verified Multi-Lingual Benchmark for Code Hallucination Detection in Fill-in-the-Middle Tasks

Anonymous ACL submission

Abstract

Large Language Models for code generation frequently produce *hallucinations* in Fill-in-the-Middle (FIM) tasks—plausible but incorrect completions such as invented API methods, invalid parameters, undefined variables, or non-existent imports. These failures pass superficial review yet introduce runtime errors. We introduce DELULU, a verified multi-lingual benchmark of **1,951** FIM samples across **7** languages and **4** hallucination types. Samples are curated through an adversarial pipeline: a frontier LLM generates plausible hallucinations, four diverse judge models evaluate them, embedding-based clustering mines progressively harder examples, self-contained Docker containers verify that golden completions compile while hallucinated variants produce the expected runtime error, and a final human-expert review removes any remaining biased or trivially decidable samples. We evaluate **11 open-weight FIM models** from five families spanning 0.5B–32B parameters: a six-point Qwen2.5-Coder scaling slate, plus a cross-family slate (CodeLlama, DeepSeek-Coder-V2, StarCoder2). The strongest model reaches only 84.5% pass@1, no family exceeds 0.77 Edit Similarity, and every family produces hallucination-aligned completions on a non-trivial share of samples, confirming that the difficulty exposed by DELULU is task-intrinsic rather than family-specific.

1 Introduction

Large Language Models (LLMs) now drive AI code assistants used by millions of developers daily. Among completion paradigms, Fill-in-the-Middle (FIM) (Bavarian et al., 2022), which generates the missing *middle* given a code prefix and suffix, has become one of the dominant approaches in production systems such as GitHub Copilot (GitHub, 2025) and Cursor (Anysphere, 2025). Yet FIM models routinely produce *hallucinations*¹: completions that are syntactically valid

and semantically plausible but factually wrong. A hallucinated method call like `df.remove_nulls()` reads naturally in a pandas context but raises an `AttributeError` at runtime; a hallucinated import from `sklearn.neural` import `DeepClassifier` extends a real namespace with a non-existent module. These failures pass superficial code review and propagate through dependency chains, introducing latent bugs that only manifest under specific runtime conditions (Lee et al., 2025).

Two complementary questions determine whether this problem is under control:

- Q1 (Generation):* Can FIM models reliably produce non-hallucinated completions across diverse languages and hallucination types?

Q2 (Detection): Can frontier LLMs reliably detect hallucinations when reviewing code completions?

If both questions could be answered positively (that models reliably avoid generating hallucinations, and that any remaining hallucinations are caught before reaching production) the problem would be effectively solved. However, the current state of the art falls short on both fronts.

Regarding *generation*, recent models have made substantial progress on established code benchmarks: top performers now exceed 90% pass@1 on HumanEval (Chen et al., 2021) and achieve competitive scores on SAFIM (Gong et al., 2024). Yet this progress is increasingly difficult to interpret. Models are suspected of *overfitting* to the fixed problem sets of popular benchmarks (Jain et al., 2024), and the dominant FIM benchmarks (HumanEval Infilling (Bavarian et al., 2022), SAFIM (Gong et al., 2024)) are mostly Python-only, lack execution ver-

to broader notions of unfaithfulness; in the code setting, taxonomies span logic, specification, and knowledge errors (Lee et al., 2025; Agarwal et al., 2024). We use the term in the *code* sense—completions that are syntactically valid and semantically plausible yet reference or rely on facts that are not true of the resolved program context.

¹“Hallucination” has multiple definitions in the literature, ranging from any factually incorrect generation (Ji et al., 2022)

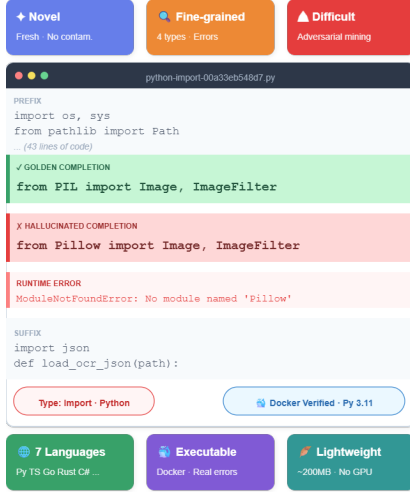


Figure 1: A DELULU sample with golden and hallucinated completions.

ification of the infilling task itself, and do not distinguish between different *types* of errors. A model that passes HumanEval Infilling may still routinely hallucinate non-existent imports or fabricate API methods when deployed on real-world code, and no current benchmark is designed to measure this.

Regarding *detection*, we ask whether a frontier LLM, acting as a code reviewer, can distinguish a correct FIM completion from a hallucinated one. To test this, we generate $\sim 40K$ FIM (prefix, suffix, completion) triples from public GitHub repositories and use CLAUDE SONNET-4.5 to produce a hallucinated variant for each golden completion across four hallucination types (method, parameter, undefined variable, import, See Table 2). We then present each completion to four frontier judge models (GPT-5.1, GPT-5.2, GPT-5.2-CODEX, GLM-4.7) and ask them to accept or reject it. We measure *both-correct accuracy*: the fraction of samples on which the judge correctly accepts the golden completion *and* correctly rejects the hallucinated one. As Table 3 (Iter 0) shows, even the best judge (GPT-5.2-CODEX) achieves only 56–90% both-correct accuracy, with import hallucinations fooling all models at rates exceeding 40%. These numbers are measured against LLM-generated labels that have not yet been execution-verified; they therefore represent a conservative lower bound on true judge capability, since some “hallucinated” labels may themselves be incorrect (Appendix B). Even so, the results confirm that hallucination detection in code remains an open problem for the most capable models available today.

Together, these findings reveal a compounding gap: frontier models cannot reliably *detect* hallucinations, and we cannot reliably *measure*

whether models generate them. What is missing is a *novel* benchmark that simultaneously provides (1) systematic hallucination categorization with fine-grained types, (2) multi-lingual coverage beyond Python, (3) execution-based verification that hallucinations produce real errors, and (4) adversarial difficulty calibration ensuring the benchmark is genuinely challenging.

We introduce DELULU² to address these requirements. DELULU is a verified multi-lingual benchmark of 1,951 FIM samples across 7 programming languages and a 4-category hallucination taxonomy (method, parameter, undefined variable, import), each producing a distinct verifiable runtime error (Figure 1). Samples are curated through a five-stage *adversarial pipeline*: hallucination generation by CLAUDE SONNET³, multi-model judging by four diverse LLMs, embedding-based clustering of judge failures to surface harder examples, difficulty-based selection with language balancing, and Docker-based execution verification. To rule out single-generator bias, every finalized sample is additionally inspected by human experts and any biased or trivially decidable rows are removed.

We then conduct what is, to our knowledge, the broadest open-weight FIM evaluation on a hallucination-targeted benchmark to date: 11 models from 5 families spanning 0.5B to 32B parameters. The Qwen2.5-Coder-Instruct (Hui et al., 2024) scaling slate (six models) achieves a maximum of 84.5% pass@1 at 32B, still failing on roughly one in seven unique FIM contexts. A cross-family slate of CodeLlama (Rozière et al., 2023), DeepSeek-Coder-V2-Lite-Instruct (DeepSeek-AI et al., 2024), and StarCoder2 (Lozhkov et al., 2024) confirms that the difficulty is family-invariant: no model exceeds 0.77 Edit Similarity, every family produces hallucination-aligned completions on 0.7–2.0% of samples, and the strongest cross-family model (DSCoder-V2-Lite-Instruct) still trails the

²The name is a play on *delulu*, internet slang for “delusional”, chosen to reflect the central property of the failure mode this benchmark targets: hallucinated completions are syntactically confident, semantically plausible, and persuasively wrong.

³We piloted three candidate generators (CLAUDE SONNET-4.5, GPT-5.1, GPT-5.2) and found CLAUDE SONNET exhibited the strongest prompt adherence, obeying the “modify exactly one element, keep everything else identical, do not signal that the output is hallucinated” constraint at the highest rate. This property is critical because the hallucinations must be statistically indistinguishable from real completions. We accordingly use CLAUDE SONNET as the sole generator; the four-judge panel and the human-verification pass below address the residual single-source bias.

Table 1: Comparison of code completion and hallucination benchmarks. DELULU uniquely combines FIM focus, hallucination categorization, execution verification, and multi-lingual coverage.

Benchmark	Task	Exec.	Langs	N	FIM	Hall.	Adv.
HumanEval (2021)	Gen.	✓	1	164	✗	✗	✗
MBPP (2021)	Gen.	✓	1	974	✗	✗	✗
MultiPL-E (2023)	Gen.	✓	18	—	✗	✗	✗
HumanEval Inf. (2022)	FIM	✗	1	1,033	✓	✗	✗
SAFIM (2024)	FIM	✗	4	17,720	✓	✗	✗
CrossCodeEval (2023)	FIM	✗	4	~10k	✓	✗	✗
DevBench (2026)	FIM	✓	6	1,800	✓	✗	✗
CodeMirage (2024)	Anal.	✗	1	1,137	✗	✓	✗
SWE-bench (2023)	SWE	✓	1	2,294	✗	✗	✗
DELULU	FIM	✓	7	1,951	✓	✓ (4)	✓

best Qwen by 1.8 pass@1 points.

Related work. Existing benchmarks fall short on the two axes DELULU targets. *Function-level* suites—HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), MultiPL-E (Casano et al., 2023), BigCodeBench (Zhuo et al., 2024), ClassEval (Du et al., 2023), CoderEval (Yu et al., 2024), and contamination-resistant variants such as LiveCodeBench (Jain et al., 2024) and EvoCodeBench (Li et al., 2024)—target generation from natural-language specifications, not FIM completion. The dominant *FIM* benchmarks (HumanEval Infilling (Bavarian et al., 2022), SAFIM (Gong et al., 2024), CrossCodeEval (Ding et al., 2023)) lack execution verification of the infilled code; DevBench (Golnari et al., 2026) executes but measures overall completion quality rather than hallucinations, and repository-level suites such as SWE-bench (Jimenez et al., 2023; Deng et al., 2025) surface hallucinations only incidentally. Recent *hallucination* work taxonomizes the failure modes (Lee et al., 2025) and analyzes patterns across model families (Agarwal et al., 2024) but ships no execution-verified benchmark artifact. Table 1 positions DELULU in this landscape; to our knowledge it is the only benchmark that simultaneously offers FIM focus, a 4-category hallucination taxonomy, multi-lingual execution verification via per-sample Docker containers, and adversarial difficulty calibration. An extended discussion is deferred to Appendix A.

Contributions.

1. A four-category FIM hallucination taxonomy, each with a distinct, execution-verifiable runtime errors (§2.1).
2. A five-stage pipeline pairing real GitHub completions with hallucinated counterparts, filtering trivial cases via frontier judges, and retaining only samples that compile as golden and provably fail as hallucinated (§2.2).
3. 1,951 execution-verified samples across 7 lan-

guages, each packaged as a self-contained Docker container with three invocation modes (§2.3).

4. Evaluation of 11 open-weight FIM models from 5 families: scaling, cross-family, per-language, and per-hallucination-type analyses providing evidence that the difficulty of DELULU generalizes beyond a single training pipeline (§4).

5. A frontier-judge study where 8 judges spanning two vendors (OpenAI and Anthropic) score paired samples; even the best (Claude-4.5-Opus) achieves 92.1% both-correct (83.4% on Import), with others as low as 52.3%—a 40-point gap showing detection on DELULU remains unsolved (§4.2).

2 Benchmark Design

DELULU is built through a five-stage pipeline (Figure 2): hallucination generation, multi-model judge evaluation, iterative adversarial sampling, difficulty-based selection, and Docker-based execution verification, followed by a human-expert review pass. We summarize the design here; full prompts, judge configurations, distributed-execution details, and the human-review protocol are in Appendix D.

2.1 Hallucination Taxonomy

DELULU targets four categories of FIM hallucinations, each defined by a distinct semantic capability and a distinct runtime error class (Table 2): **Method** (invented method names $\Rightarrow$ AttributeError), **Parameter** (non-existent key-word arguments $\Rightarrow$ TypeError), **Undefined Variable** (out-of-scope references $\Rightarrow$ NameError), and **Import** (non-existent modules $\Rightarrow$ ImportError). Every hallucination is constrained to be *plausible*—following the naming conventions of the surrounding API rather than appearing obviously synthetic—so that detection cannot rely on surface cues.

The four categories are a deliberate subset of the refined code-hallucination taxonomy of Gao et al. (2025), which catalogues API Misuse, Library Misuse, and Dependency Hallucination as the primary knowledge-level failure modes in code LLMs. We retain exactly the categories that (i) occur frequently in FIM completion, (ii) map to a small, well-defined set of runtime error classes, and (iii) can therefore be automatically verified by executing the assembled file. Other hallucination modes documented in the literature—logic-flow violations, deprecated-API usage, off-by-one errors, behavior-correct-but-API-wrong completions, and broader functional misalignment (Lee et al., 2025; Gao et al., 2025)—are excluded by design: their ground truth depends on project-specific test suites,

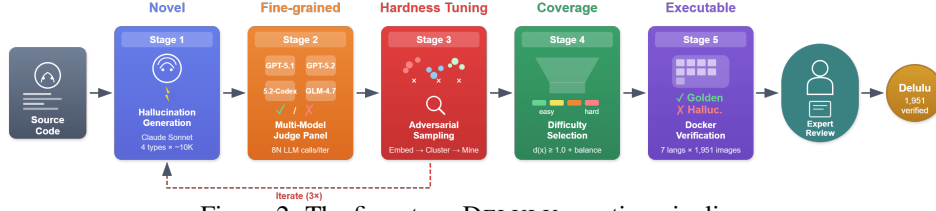


Figure 2: The five-stage DELULU curation pipeline.

semantic-equivalence judges, or human raters, each of which would reintroduce the label noise that the execution gate is meant to eliminate. DELULU should accordingly be read as a *lower bound* on hallucination prevalence: failure on a DELULU sample implies a production-relevant error, but passing DELULU does not preclude semantic hallucinations of the excluded types. Extending coverage to non-error categories is the principal item on our roadmap (Appendix K). The four categories share a single-element-modification protocol: the golden and hallucinated tuples differ in exactly one method name, one keyword argument, one identifier, or one import path; prefix, suffix, surrounding identifiers, and formatting are all identical. Figure 1 shows a representative Import sample illustrating this protocol; More examples are reproduced in Appendix L.

2.2 Curation Pipeline

The pipeline’s goal is simple: produce a large pool of (golden, hallucinated) FIM pairs, then keep only those that are *both* non-trivial—hard enough that frontier code reviewers cannot dismiss them at a glance—and executable, so the hallucinated variant provably fails at runtime. To achieve this, we alternate three operations across iterations: (i) generate candidate pairs from real GitHub source, (ii) probe the candidates with a panel of frontier judges to surface failure modes, and (iii) mine new candidates that resemble those failure modes. Each iteration thus tightens the difficulty distribution while leaving easy or already-saturated cases behind.

Starting from source files scraped from public GitHub repositories, CLAUDE SONNET generates paired golden and hallucinated completions using type-specific prompts ($N \approx 10K$ samples per type per iteration); we use CLAUDE SONNET as the sole generator because a pre-experiment found it had the strongest prompt adherence among candidate frontier models (see footnote in §1). Four judges (GPT-5.1, GPT-5.2, GPT-5.2-CODEX, GLM-4.7) then independently score each (sample, completion) pair as correct/incorrect with chain-of-thought reasoning, yielding $8N$ LLM calls per iteration. We analyze judge disagreements, embed their reasoning with all-MiniLM-L6-v2, cluster via K-means with

Table 2: Hallucination taxonomy.

Type	Error Class	Semantic Req.
Method	AttributeError	API method knowledge
Parameter	TypeError	Signature knowledge
Undef. Var.	NameError	Scope tracking
Import	ImportError	Package ecosystem

Table 3: Both-correct accuracy across adversarial iterations (Iter 0 / 1 / 2). Decreasing accuracy in most cases confirms harder samples are mined over time.

Judge	Import	Method	Param.	Undef.
GLM-4.7	.17/.04/.03	.31/.16/.22	.23/.17/.28	.42/.21/.50
GPT-5.1	.52/.40/.55	.64/.55/.52	.57/.58/.56	.78/.76/.77
GPT-5.2	.52/.43/.54	.62/.65/.52	.56/.55/.56	.74/.71/.72
GPT-5.2-CODEX	.56/.48/.59	.77/.76/.67	.65/.64/.64	.90/.89/.89

silhouette-based k selection, and mine semantically similar code from the unlabeled corpus (cosine similarity in $[0.75, 0.99]$). This loop runs for 3 iterations. The mining loop is validated by Table 3: judge accuracy decreases across iterations, confirming that progressively harder samples are surfaced. GLM-4.7 degrades most sharply on import ($0.17 \rightarrow 0.04 \rightarrow 0.03$), reflecting that open-weights models are particularly vulnerable to ecosystem-specific hallucinations; GPT-5.2-CODEX maintains ≥ 0.89 on undefined variable, reflecting the relative simplicity of scope reasoning. We note that some GPT-family judges show non-monotonic import accuracy across iterations (e.g., GPT-5.1: $0.52 \rightarrow 0.40 \rightarrow 0.55$). This does not contradict the overall trend: the adversarial loop mines *different* failure modes in each iteration rather than monotonically scaling a single difficulty axis. In Iter. 2, the mined import samples happen to overlap less with the failure patterns of some GPT judges, causing a partial recovery. The key validation signal is the *cross-judge average*, which decreases monotonically when pooled across all judges.

Difficulty selection. We score each sample by $d(x) = \sum_j \mathbb{I}[\text{judge}_j(x_{\text{hall}}) = 1] + 0.5 \sum_j \mathbb{I}[\text{judge}_j(x_{\text{gold}}) = 0]$ and require $d(x) \geq 1.0$. This equation essentially filters out samples that could not fool enough models. Language balancing targets $\lfloor N/7 \rfloor$ samples per language.

2.3 Execution Verification

Every released sample is **execution-verified**: ground-truth labels do not rely on heuristics or LLM judgments. For each sample we assemble

two complete source files (prefix + completion + suffix) and run them inside a per-language Docker container. The golden file must compile/parse and run without errors; the hallucinated file must produce the *specific* error class corresponding to its hallucination category (e.g. `AttributeError`, `TypeError`, ...). Samples failing either condition are discarded. Verification rests on three components, summarized below; full details are in § H.

Per-language base images. We maintain seven base images covering Python (CPython 3.11), Java (OpenJDK 17), TypeScript (Node.js 20), Go (1.21), Rust (nightly), C# (.NET 8.0), and C++ (GCC 13/Clang). Each ships with the standard package manager plus a dependency-resolution module that installs the source file’s imports before verification.

Standalone-file precheck. Each DELULU sample is a single source file mined from a large open-source project, and most files in such projects depend transitively on sibling files in the same repository. To keep every per-sample container lightweight and self-contained, a static precheck discards files whose dependency graph is not closable within a single file. The precheck retains roughly 5% of mined files and is the dominant determinant of overall yield.

Docker verification with a CLAUDE SONNET-assisted fix loop. Conditional on passing the precheck, ~ 0.4 of the (golden, hallucinated) tuples verify on the first attempt. For the remainder, a CLAUDE SONNET-assisted fix loop (up to three iterations) inspects the failure trace and proposes patches to the container environment—installing missing system libraries, adding package dependencies, or adjusting build flags—raising the verification rate to ~ 0.75 on standalone files. Successfully verified samples are baked into self-contained Docker containers in Dockerhub exposing three modes: `verify golden`, `verify hallucinated`, and `verify patch` (which evaluates a model-generated completion supplied via stdin), enabling drop-in integration with any FIM evaluation pipeline.

2.4 Human Expert Review

After execution verification, every Docker-verified sample is reviewed by a panel of three human experts via a dedicated annotation interface (Figure 5 in Appendix E). The interface shows reviewers the prefix, suffix, and *both* completions *labelled* as golden and hallucinated—so the task is not to guess which is which but to validate that the labels are correct and the pair is non-trivially distinguish-

able. Each expert then selects one of three actions: *accept* (the tuple is valid as-is), *reject* (the tuple is fundamentally flawed), or *edit* (the hallucinated completion is modified to better fit the context). Edited samples are re-run through the Docker execution pipeline to confirm that the revised hallucinated completion still triggers the expected runtime error. Of 1,957 Docker-verified candidates, 1,744 were accepted as-is, 6 were rejected, and 207 were edited and re-verified, yielding the final 1,951 released samples. This post-verification human pass is what guarantees the benchmark is free from residual generator bias and label noise.

3 Benchmark Statistics

Distribution. The final DELULU dataset contains 1,951 execution-verified samples distributed across seven programming languages and all four hallucination types; because some FIM contexts share a prefix, suffix, and golden completion across more than one hallucinated variant, these 1,951 samples cover 950 unique FIM contexts. Table 4 shows the joint distribution. TypeScript contributes the most samples (420, 21.5%), followed by Python (374, 19.2%); undefined variable is the most frequent category (577) and parameter the least (435).

Two cells stand out and are expected rather than incidental. `Python×Parameter` is small (22) because dynamically typed Python functions often accept arbitrary keyword arguments without raising `TypeError`, so very few mined Python sites admit a clean parameter hallucination. C++ contributes the fewest samples per language (125) because the standalone-file precheck (§2.3) retains far fewer C++ files than other languages: most C++ source files in real repositories rely on headers and translation units that live in sibling files. As a general rule, per-(language, type) cells below ~ 50 samples should be read as suggestive rather than confirmatory. To address the imbalance directly, we are actively curating an extended release in which every cell contains at least 100 verified samples; the present paper releases the current 1,951-sample artifact and reports the extended-release timeline on the project page.

Provenance and Freshness. Source code for the benchmark is drawn from a code-mining pipeline that extracts API call sites from public GitHub repositories. The pipeline targets ~ 25 third-party packages per language, selected by adoption frequency in anonymized code completion telemetry, LLM-based ecosystem ranking, and expert endorsement. For the current version, mined in

Table 4: Sample distribution: language $\times$ hall. type.

Lang.	Imp.	Meth.	Par.	Und.	Tot.
C++	24	32	35	34	125
C#	26	51	82	87	246
Go	50	81	79	81	291
Java	44	59	65	75	243
Python	166	78	22	108	374
Rust	57	60	66	69	252
TS	111	100	86	123	420
Total	478	461	435	577	1951

October 2025, target package versions span April–September 2025, meaning the source repositories had adopted recent library releases at the time of collection. C++ libraries are selected via expert curation rather than version constraints. The 1,951 pre-hallucination examples span 319 repositories (median 81 GitHub stars) and 510 source files.

Complexity. Beyond the hallucination axis, a benchmark’s difficulty also depends on the structural complexity of the code surrounding each completion: a model that handles short, self-contained snippets may still fail once the surrounding code grows longer or its control flow becomes more intricate. We therefore characterize each DELULU sample along four *code-complexity* dimensions and compare DELULU to existing FIM benchmarks on the same axes (Table 5): *prefix lines of code* (Pfx LOC), the amount of context the model reads before the hole; *total LOC* (Tot LOC), the length of the fully assembled source file; *completion tokens* (Cmp Tok.), the output length measured via `cl100k_base`; and *cyclomatic complexity* (CC) (Landman et al., 2016), the number of linearly independent paths through the code, which proxies the control-flow reasoning required to fill the hole. We can observe in Table 5 that DELULU has the highest cyclomatic complexity among all compared benchmarks (16.4, roughly $2\times$ SAFIM and $3\times$ DevBench), because samples are drawn from real-world GitHub repositories with non-trivial branching, looping, and error handling. It also has the highest total file length among benchmarks whose completions span more than one line (152.6 LOC vs. 65.3 for DevBench), meaning models must reason over substantially more surrounding context. Per-language complexity varies: C# samples average 255.3 LOC and 22.1 CC (enterprise-style code), while TypeScript averages 77.0 LOC and 9.6 CC (compact web-development patterns). The full per-language breakdown is in Appendix I.

4 Evaluation

Models. We evaluate two open-weight FIM slates totaling 11 models from 5 families. The *primary slate* is the six-point Qwen2.5-Coder-Instruct

Table 5: Complexity comparison across FIM BMs.

Benchmark	N	Pfx LOC	Tot LOC	Cmp Tok.	CC
HumanEval Inf.	409	20.1	21.8	17.4	4.2
DevBench	1,800	41.4	65.3	44.2	5.5
SAFIM	22,291	65.4	66.3	18.2	8.1
Real-FIM-Eval	17,879	222.1	222.1	—	6.1
DELULU	1,951	61.7	152.6	15.8	16.4

family (Hui et al., 2024) (0.5B, 1.5B, 3B, 7B, 14B, 32B), chosen for its standardized FIM tokens and broad parameter range. The *cross-family slate* adds five FIM-capable models from four independent training pipelines: CodeLlama-7B/13B (Rozière et al., 2023), DeepSeek-Coder-V2-Lite-Instruct (16B MoE) (DeepSeek-AI et al., 2024), StarCoder2-7B/15B (Lozhkov et al., 2024).

Setup. All models use greedy decoding (temperature = 0) with a maximum output length of 256 tokens, run on Azure ML batch endpoints over the full 1,951 samples. For base FIM models without an instruction-tuned stop boundary (StarCoder2, CodeLlama), we apply the line-count truncation convention used by HumanEval-Infilling (Bavarian et al., 2022) and SAFIM (Gong et al., 2024), truncating each prediction to the gold completion’s line count before scoring. The smallest instruct model (Qwen2.5-Coder-0.5B-Instruct) and base StarCoder2 frequently emit degenerate continuations (e.g. repeated boilerplate, or runs that never emit an end-of-completion marker) so we additionally cap their decoding at the gold completion’s line budget; this prevents a single degenerate generation from cascading into spurious compile errors that would otherwise be charged against the model. Because some FIM contexts are paired with multiple hallucination variants during curation, the 1,951 samples cover 950 unique (prefix, suffix, golden) tuples, and a model only sees the FIM context—identical predictions are emitted for all siblings of a unique context. We therefore report **pass@1 over the 950 unique contexts** (one representative per group) as the headline metric throughout this section; the per-sample view differs by less than 0.02 absolute points on every model and the family ranking is unchanged. Alongside pass@1 we report four static metrics on every prediction: **EM** (Exact Match) is the fraction of completions identical to the gold completion byte-for-byte; **ES** (Edit Similarity) is character-level normalized Levenshtein similarity to the gold; **CB** (CodeBLEU) (Ren et al., 2020) weights n -gram, AST, and dataflow agreement using language-specific parsers; and **HR** (Hallucination Rate) is the share of predictions whose SequenceMatcher similarity

Table 6: Results on DELULU for all 11 evaluated models. EM, ES, CB, HR over 1,951 samples; **pass@1** over 950 unique FIM contexts. **Bold** = best; underline = second.

Model	Params	EM↑	ES↑	CB↑	HR↓	pass@1↑
Qwen2.5-Coder-0.5B	0.5B	.228	.625	.360	.020	.436
Qwen2.5-Coder-1.5B	1.5B	.378	.632	.423	.016	.647
Qwen2.5-Coder-3B	3B	.440	.693	.472	.017	.767
Qwen2.5-Coder-7B	7B	.402	.625	.472	.011	.711
Qwen2.5-Coder-14B	14B	.462	.716	.515	.010	.815
Qwen2.5-Coder-32B	32B	.541	.765	.531	.010	.845
CodeLlama-7B	7B	.468	.695	.463	.017	.711
CodeLlama-13B	13B	<u>.473</u>	.701	.478	.013	.719
DSCoder-V2-Lite	16B*	.469	<u>.717</u>	.489	.011	<u>.827</u>
StarCoder2-7B	7B	.097	.596	.445	.013	.193
StarCoder2-15B	15B	.215	.600	.466	.007	.372

All instruction-tuned. *MoE: 2.4B active params/token.

to the hallucinated variant exceeds that to the gold. Formal definitions are in Appendix J. Static metrics (EM, ES, CB, HR) are reported at the per-sample level since they are computed on each prediction.

4.1 Results

Table 6 presents the unified results for all 11 models and Figure 3 shows the Qwen2.5-C scaling trends.

Scaling behavior (Qwen2.5-Coder). Four observations stand out within the primary slate. (1) Scaling improves pass@1 from 43.6% (0.5B) to 84.5% (32B), a 41-point range. (2) At 0.5B, text quality is already a clear bottleneck: even after the line-budget cap, only 22.8% of completions match the gold byte-for-byte and Edit Similarity is 0.625, well below every other Qwen size; 1.5B closes most of that gap (37.8% EM, 0.632 ES, 64.7% pass@1). (3) 3B achieves 76.7% pass@1 versus 7B’s 71.1%, with the 7B model showing *lower* Edit Similarity (0.625 vs. 0.693) and Exact Match (0.402 vs. 0.440) than 3B; additional parameters in this range do not translate to better hallucination avoidance. (4) Even at 32B, 15.5% of unique contexts still fail. By hallucination type, import is hardest at every scale (77.1% for 32B vs. 85.1–90.6% on the other three categories), reflecting that detecting fake imports requires memorized knowledge of the package ecosystem rather than local pattern completion. Per-language and per-language- $\times$ -type breakdowns are in Figure 3 and Appendix J. The 3B>7B inversion is reproducible across all four metrics in our single-seed run, but we do not have sufficient evidence to attribute it to a specific cause. The proximate observation is that 7B variant over-generates on FIM contexts where the 3B variant stops cleanly, but identifying the underlying cause would require multi-seed verification, layer-wise probing, and access to training-recipe details. We therefore report the inversion as an artifact of a single family on this benchmark, do not generalize from it, and list a deeper investigation as an explicit open question (Appendix K).

Cross-family Comparison. To verify that the dif-

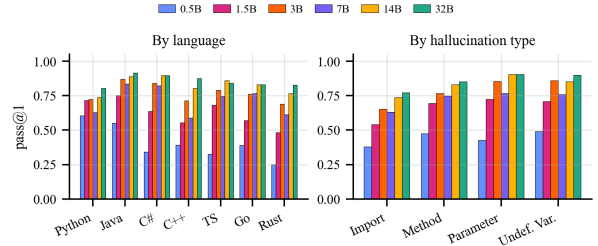
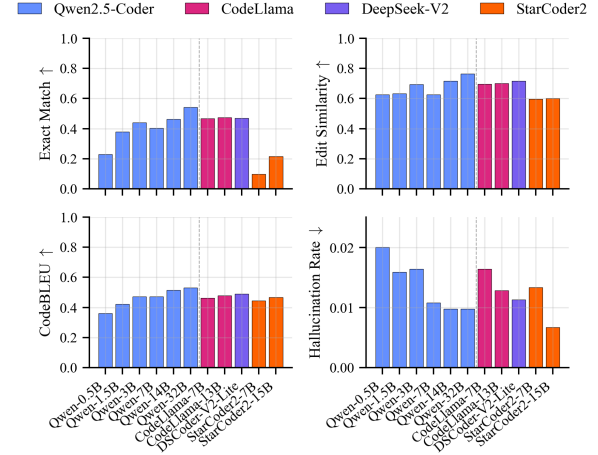
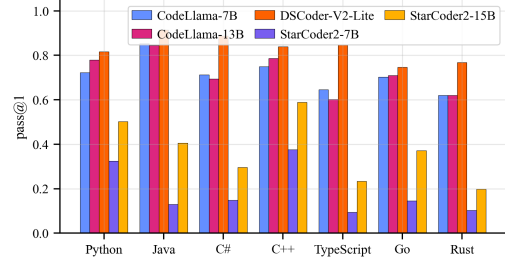


Figure 3: Qwen2.5-Coder scaling on DELULU: pass@1 by language (left) and hallucination type (right).



(a) Cross-family static metrics on DELULU.



(b) Per-language pass@1 for the cross-family slate.

iculty exposed by DELULU is intrinsic rather than an artifact of the Qwen2.5-Coder pretraining recipe, we examine the cross-family rows of Table 6 alongside Figure 4a.

Four observations. (1) **The benchmark is hard for every family we tested, not just Qwen.** If DELULU were merely surfacing a Qwen-specific weakness, models from other training pipelines should clear it easily. They do not: across CodeLlama, DeepSeek-Coder-V2, and StarCoder2, no cross-family model exceeds 0.72 Edit Similarity or 0.49 CodeBLEU—essentially the same ceiling reached by the best Qwen-14B/32B checkpoints—so the difficulty appears to be a property of the task rather than of any single model lineage. (2) **Hallucination is universal.** Every model surveyed produces hallucination-aligned completions on 0.7–2.0% of samples; StarCoder2-15B has the lowest similarity-based HR (0.007) yet still trails Qwen-32B’s Edit Similarity by 16.5 absolute points.

(3) **Instruction tuning helps the model know when to stop, not what to write.** Base FIM models such as StarCoder2 frequently keep generating past the gold completion, emitting trailing tokens that are syntactically reasonable but extend beyond the intended hole; this overshoot is penalized harshly by Exact Match (which requires byte-level equality with the gold completion) but barely affects Edit Similarity and CodeBLEU, which tolerate small extra suffixes. The instruction-tuned families (Qwen-Instruct, DSCoder-V2-Lite-Instruct) therefore score much higher on Exact Match than StarCoder2 yet differ by less than three absolute points on Edit Similarity and CodeBLEU. The capability gap that instruction tuning closes is boundary detection (where the completion ends), not the underlying knowledge needed to avoid hallucinating an identifier in the first place—which is precisely what DELULU targets. (4) **pass@1 spans a wide range.** DSCoder-V2-Lite-Instruct is the strongest cross-family model (0.827); CodeLlama-7B/13B cluster at 0.711–0.719; base StarCoder2 drops to 0.19–0.37. Even the best cross-family pass@1 trails Qwen-32B’s 0.845 by 1.8 points. Per-language pass@1 (Figure 4b) shows Java is uniformly easiest (≥ 0.85 on every instruction-tuned cross-family model) and Rust the hardest, mirroring the within-family pattern.

4.2 DELULU for Hallucination Detection

Beyond measuring whether FIM models *generate* hallucinations, DELULU can also measure whether frontier models can *detect* them. Given a completion and its surrounding context, can a general-purpose LLM determine whether the completion is correct? This reframes the same samples as a binary classification task and lets us test whether the difficulty ordering observed in generation (§4.1), where import hallucinations are hardest, persists when the task shifts from generation to verification. Unlike the Iter 0 judge evaluation used during curation (Table 3), which was measured against unverified LLM-generated labels, the experiment below evaluates judges against DELULU’s execution-verified and human-reviewed ground truth.

Protocol. We present each of the 1,951 samples to a judge model twice: once with the golden completion inserted between the prefix and suffix, and once with the hallucinated completion. The judge uses chain-of-thought reasoning and returns a binary score (1 = correct, 0 = hallucinated). A sample is *both-correct* only if the judge accepts the golden completion *and* rejects the hallucinated one. We

Table 7: LLM-as-judge hallucination detection on DELULU. Gold/Hall = accept golden / reject hallucinated; Both = conjunction. Imp/Meth/Par/Und = Both by type. **Bold** = best; underline = second.

Judge	Gold $\uparrow$	Hall $\uparrow$	Both $\uparrow$	Imp	Meth	Par	Und
GPT-4o-mini	.791	.716	.523	.487	.546	.461	.580
GPT-4.1-mini	.869	.803	.677	.569	.730	.635	.755
GPT-5.4-mini	.894	.849	.748	.692	.782	.663	.831
Claude-4.5-Haiku	.941	.836	.779	.656	.771	.743	.915
GPT-5.4	.962	.865	.828	.679	.842	.798	.946
Claude-4.6-Sonnet	.988	.870	.858	.803	.784	.885	.943
GPT-5.2-Codex	.969	.912	.882	.754	<u>.927</u>	.872	.962
Claude-4.5-Opus	.982	.938	.921	.834	.946	.911	.981

report the conjunction rather than only marginals because each gold/hallucinated pair shares a single source file: a constant-bias judge achieves a high marginal on one direction but zero on the conjunction. Precision, Recall, F1, and MCC are in Appendix C; the judge prompt is in Appendix G.

Results. Table 7 reports results for 8 judge models spanning two vendors (OpenAI and Anthropic), selected to form comparable pairs across capability tiers. Three findings. (1) **Even the strongest model leaves a gap.** Claude-4.5-Opus leads at 92.1% both-correct, followed by GPT-5.2-Codex (88.2%); no model fully solves the benchmark. Smaller models such as GPT-4o-mini (52.3%) and GPT-4.1-mini (67.7%) score far lower, confirming that the benchmark discriminates across a 40-point capability range. (2) **The difficulty ordering is vendor-invariant.** Import hallucinations are the hardest category for every model from both vendors, while undefined-variable hallucinations are consistently the easiest, mirroring the generation results from §4.1. Claude-4.5-Opus reaches 83.4% on imports yet 98.1% on undefined variables; GPT-5.2-Codex shows a similar 21-point gap (75.4% vs. 96.2%). (3) **The bottleneck is hallucination rejection, not golden acceptance.** Stronger judges achieve $\geq 89\%$ golden acceptance, but even Claude-4.5-Opus misses $\sim 6\%$ of hallucinated completions. This confirms that the package-ecosystem knowledge required to distinguish real from fake imports is a genuine capability gap that persists across model families, vendors, and task framing. Per-language detection results are in Appendix B.

5 Conclusion

We introduced DELULU, a multi-lingual, execution-verified benchmark for evaluating hallucination in FIM code completion. Experiments across 11 models and multiple judges show that hallucination remains a persistent, structured failure mode. DELULU provides a unified testbed for studying and mitigating hallucination, highlighting the need for approaches beyond scaling, such as stronger grounding and execution-aware verification.

6 Limitations

- **Sample size.** The released artifact contains 1,951 samples (~ 280 per language, ~ 70 per (language, type) cell on average); cells below ~ 50 samples (notably Python $\times$ Parameter at 22 and C++ at 125 overall) carry visibly wider sampling intervals and per-cell numbers should be read as suggestive rather than confirmatory. An extended release in which every (language, hallucination type) cell contains at least 100 verified samples is currently in curation.
- **Standalone-file constraint.** To keep each per-sample Docker container lightweight, we admit only source files whose static dependency graph is closable within the file itself (§2.3). Samples that genuinely require multi-file or repository-level context are therefore under-represented; DELULU complements rather than replaces repository-level FIM benchmarks such as CrossCodeEval (Ding et al., 2023).
- **Hallucination taxonomy.** The 4 categories cover the most common knowledge-level FIM hallucination modes from Gao et al. (2025) but exclude logic errors, type-mismatch errors that compile silently, deprecated-API usage, behavior-correct-but-API-wrong completions, and other semantic-correctness failures that do not raise a categorical runtime error. DELULU should accordingly be read as a *lower bound* on hallucination prevalence (§2.1).
- **Completion length.** Completions average 2.0 LOC, reflecting the typical granularity of localized FIM hallucinations targeted by our taxonomy. Multi-statement and block-level hallucinations would test different aspects of hallucination resilience and are not in scope for this release.
- **Temporal validity.** Package ecosystems evolve; some hallucinated imports may become valid (or golden ones may be deprecated) over time. The released artifact is pinned to the October 2025 mining snapshot; we plan periodic refreshes alongside the extended release.

7 Ethical considerations

- **Intended use.** DELULU is released as a *test-only* artifact with no pre-defined train/validation split. Fine-tuning on DELULU samples is explicitly discouraged. Researchers needing a training source

for hallucination mitigation should regenerate samples through the released pipeline against a different seed corpus.

- **Misuse.** The paired golden/hallucinated structure could in principle be used to fine-tune models that produce *more* convincing hallucinations. We consider this risk modest relative to the benefit of measurable hallucination evaluation, and we mitigate it operationally by releasing only the curated benchmark (not the intermediate generation/judging pipeline outputs) and by recommending that any defensive fine-tuning on DELULU be reported transparently.
- **Data licensing and provenance.** The DELULU dataset follows a collection-of-licenses model. While the benchmark codebase itself is released under an MIT license, each individual dataset entry is derived from publicly available GitHub repositories and remains governed by the original license of its source repository. For transparency and compliance, every sample includes metadata specifying its SPDX-style license identifier and, where available, the originating repository URL. As a result, downstream users must adhere to the terms of the corresponding upstream license for each sample, including any applicable attribution, copyleft, or share-alike requirements.

References

- Vibhor Agarwal, Yulong Pei, Salwa Alami, and Xiaomo Liu. 2024. [Codemirage: Hallucinations in code generated by large language models \(2024\)](#). *arXiv.org*.
- Anysphere. 2025. Cursor: The AI Code Editor. <https://www.cursor.com>. Accessed: 2025-06-01.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, H. Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and 1 others. 2021. Program synthesis with large language models. *arXiv.org*.
- Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. 2022. [Efficient training of language models to fill in the middle](#). *arXiv.org*.
- Federico Cassano, John Gouwar, Daniel Nguyen, S. Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q. Feldman, and 1 others. 2023. [Multipl-e: A scalable and polyglot approach to benchmarking neural code generation](#). *IEEE Transactions on Software Engineering*, 49(7):3675–3691.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé, Jared Kaplan, Harrison Edwards, Yura Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv.org*.
- DeepSeek-AI, Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y. Wu, Yukun Li, Huazuo Gao, and 1 others. 2024. [DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence](#). *arXiv.org*.
- Xiang Deng, Jeff Da, Edwin Pan, Yan He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, and 1 others. 2025. [Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?](#) *arXiv.org*.
- Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, M. K. Ramathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and 1 others. 2023. [Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion](#). *Neural Information Processing Systems*, pages 46701–46723.
- Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2023. [Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation](#). *arXiv.org*.
- Sujan Dutta, Sayantan Mahinder, Raviteja Anantha, and Bortik Bandyopadhyay. 2024. [Applying rlaif for code generation with api-usage in lightweight llms](#). In *NLRSE*, pages 39–45. Association for Computational Linguistics.
- Cuiyun Gao, Guodong Fan, Chun Yong Chong, Shizhan Chen, Chao Liu, David Lo, Zibin Zheng, and Qing Liao. 2025. [A systematic literature review of code hallucinations in LLMs: Characterization, mitigation methods, challenges, and future directions for reliable AI](#). *arXiv.org*.
- GitHub. 2025. GitHub Copilot. <https://github.com/features/copilot>. Accessed: 2025-06-01.
- Pareesa Ameneh Golnari, Adarsh Kumarappan, Wen Wen, Xiaoyu Liu, Gabriel Ryan, Yuting Sun, Shengyu Fu, and Elsie Nallipogu. 2026. [Devbench: A realistic, developer-informed benchmark for code generation models](#). *arXiv.org*.
- Linyuan Gong, Sida Wang, Mostafa Elhoushi, and Alvin Cheung. 2024. [Evaluation of LLMs on syntax-aware code fill-in-the-middle tasks](#). *International Conference on Machine Learning*, (arXiv:2403.04814). ArXiv:2403.04814 [cs].
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, D. Song, and 1 others. 2021. Measuring coding challenge competence with apps. *NeurIPS Datasets and Benchmarks*, 34.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, K. Dang, and 1 others. 2024. Qwen2.5-coder technical report. *arXiv.org*.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. [Live-codebench: Holistic and contamination free evaluation of large language models for code](#). *International Conference on Learning Representations*.
- Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, D. Su, Yan Xu, Etsuko Ishii, Yejin Bang, Delong Chen, Wenliang Dai, and 1 others. 2022. [Survey of hallucination in natural language generation](#). *ACM Computing Surveys*, 55(12):1–38.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. [Swe-bench: Can language models resolve real-world github issues?](#) *International Conference on Learning Representations*.
- D. Landman, Alexander Serebrenik, and J. Vinju. 2016. [Empirical analysis of the relationship between cc and sloc in a large corpus of java methods and c functions](#). *J. Softw. Evol. Process.*, 28(7):589–618.
- Yunseo Lee, John Youngeun Song, Dongsun Kim, Jinda Kim, Mijung Kim, and Jaechang Nam. 2025. [Hallucination by code generation llms: Taxonomy, benchmarks, mitigation, and challenges](#). *arXiv.org*.
- Jia Li, Ge Li, Xuanming Zhang, Yihong Dong, and Zhi Jin. 2024. [Evocodebench: An evolving code generation benchmark aligned with real-world code repositories](#). *arXiv.org*, abs/2404.00599.

- Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, J. Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, and 1 others. 2024. [StarCoder 2 and the stack v2: The next generation](#). *arXiv.org*.
- Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, M. Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv.org*.
- Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Tan, Yossi Adi, Jingyu Liu, Tal Remez, J. Rapin, and 1 others. 2023. [Code Llama: Open foundation models for code](#). *arXiv.org*.
- Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. 2024. [Codereval: A benchmark of pragmatic code generation with generative pre-trained models](#). *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–12.
- Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, and 1 others. 2024. [Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions](#). *International Conference on Learning Representations*.

A Extended Related Work

Function-level benchmarks. HumanEval (Chen et al., 2021) introduced functional-correctness evaluation with 164 hand-crafted Python programming problems; MBPP (Austin et al., 2021) scaled this to 974 crowd-sourced problems; APPS (Hendrycks et al., 2021) added competitive-programming difficulty. These suites evaluate standalone function generation from natural-language descriptions—a substantially different task from FIM completion—and are Python-only and prone to training-data contamination (Jain et al., 2024).

Multi-lingual benchmarks. MultiPL-E (Cassano et al., 2023) transpiles HumanEval and MBPP to 18 languages; BigCodeBench (Zhuo et al., 2024) evaluates diverse function calls across 139 libraries; ClassEval (Du et al., 2023) tests class-level generation; CoderEval (Yu et al., 2024) focuses on pragmatic generation with real-world dependencies. All remain focused on code *generation* from specifications rather than FIM *completion*.

Contamination-resistant benchmarks. LiveCodeBench (Jain et al., 2024) uses time-stamped competitive-programming problems; EvoCodeBench (Li et al., 2024) provides evolving evaluation aligned with real repositories. DELULU complements these via fresh-source curation and adversarial mining.

FIM benchmarks (extended). HumanEval Infilling (Bavarian et al., 2022) adapts HumanEval to FIM with single-line, multi-line, and random-span settings, but is Python-only and lacks execution verification of the infill. SAFIM (Gong et al., 2024) introduces syntax-aware FIM categories (block, API, control flow) across four languages, but does not verify execution. CrossCodeEval (Ding et al., 2023) evaluates cross-file completion in repository contexts. DevBench (Golnari et al., 2026) provides telemetry-driven FIM evaluation across six languages with execution and detailed diagnostics, but measures general completion quality rather than targeting hallucinations. Repository-level suites SWE-bench (Jimenez et al., 2023; Deng et al., 2025) expose hallucination-prone scenarios but cannot attribute failures to specific hallucination categories.

Code hallucination (extended). Lee et al. (2025) provide a comprehensive taxonomy of hallucinations in LLM-based code generation; CodeMi-

rage (Agarwal et al., 2024) analyzes hallucination patterns across model families. Both establish that hallucinations are common but provide no execution-verified benchmark. Existing detection approaches include static-analysis post-hoc verification, LLM-as-judge evaluation (Dutta et al., 2024), and retrieval-augmented generation. Our work uses a panel of four diverse LLM judges during *curation*—not evaluation—and validates the difficulty calibration via judge-accuracy decay across iterations.

B Initial Judge Evaluation (Iter 0) and Label-Noise Considerations

Table 3 (Iter 0 column) reports the both-correct accuracy of four frontier judge models on the initial $\sim 40K$ LLM-generated FIM hallucination pairs, before any adversarial mining, execution verification, or human review.

Protocol. We sampled real-world source files from public GitHub repositories, extracted FIM (prefix, suffix, completion) triples, and used CLAUDE SONNET to generate a hallucinated variant for each golden completion across the four hallucination types. Four judges (GPT-5.1, GPT-5.2, GPT-5.2-CODEX, GLM-4.7) then independently scored each (sample, completion) pair as correct/incorrect with chain-of-thought reasoning. A sample is *both-correct* only if the judge accepts the golden completion *and* rejects the hallucinated one.

Key findings. (1) Even the best judge (GPT-5.2-CODEX) achieves only 56–90% both-correct accuracy, meaning it either accepts a hallucinated completion or rejects a valid one on 10–44% of samples. (2) **Import is hardest to detect:** every judge scores below 56% on imports, confirming that distinguishing real from fake package/module names requires ecosystem knowledge that even frontier models lack. GLM-4.7 drops to 17%. (3) **Undefined variable is easiest:** scope violations are locally verifiable, and GPT-5.2-CODEX reaches 90%. (4) There is a large gap between GLM-4.7 (17–42%) and the GPT-family judges (52–90%).

Label-noise caveat. Crucially, these Iter 0 numbers are measured against *LLM-generated labels that have not been execution-verified*. Since CLAUDE SONNET’s hallucination generation is imperfect, some samples labeled “hallucinated” may in fact be valid code (and vice versa). When a judge “incorrectly” accepts such a sample, it may actually

be right; the noisy label penalizes it. The Iter 0 accuracy therefore represents a *conservative lower bound* on true judge capability. This is precisely why Docker-based execution verification (Stage 5, §2.3) and human expert review are essential: they eliminate label noise before the benchmark is finalized. The formal detection experiment on the verified DELULU dataset (§4.2) measures judge capability against execution-grounded labels and should be interpreted independently of these Iter 0 numbers.

C Detection: Precision, Recall, F1, and MCC

The main-paper detection results (§4.2, Table 7) report golden acceptance, hallucination rejection, and their conjunction (both-correct accuracy). Some readers may prefer single-direction summary statistics; we therefore re-express the same per-judge confusion matrices as Precision, Recall, F1, and Matthews Correlation Coefficient (MCC), treating *hallucination* as the positive class.

Let GA denote golden acceptance and HR denote hallucination rejection on the $N=1,951$ paired evaluations per judge. Treating each judge’s 0 output (“reject as hallucinated”) as a positive prediction, the four confusion-matrix counts are $TP=N \cdot HR$, $FN=N(1-HR)$, $FP=N(1-GA)$, and $TN=N \cdot GA$, from which Precision, Recall, F1, and MCC follow directly. Table 8 summarizes the resulting metrics.

Table 8: Single-direction detection metrics for the eight judges of Table 7, derived from their Gold Acc. and Hall Rej. rates with “hallucination” as the positive class. **P**: Precision; **R**: Recall ($\equiv$ Hall Rej.); **F1**: harmonic mean of P and R; **MCC**: Matthews Correlation Coefficient. **Bold** = best in column.

Vendor	Judge	P $\uparrow$	R $\uparrow$	F1 $\uparrow$	MCC $\uparrow$
OpenAI	GPT-4o-mini	.774	.716	.744	.508
OpenAI	GPT-4.1-mini	.860	.803	.830	.673
OpenAI	GPT-5.4-mini	.889	.849	.869	.744
Anthropic	Claude-4.5-Haiku	.934	.836	.882	.781
OpenAI	GPT-5.4	.958	.865	.909	.831
Anthropic	Claude-4.6-Sonnet	.986	.870	.925	.864
OpenAI	GPT-5.2-Codex	.967	.912	.939	.882
Anthropic	Claude-4.5-Opus	.981	.938	.959	.921

Three observations. (1) **Precision is uniformly high for the stronger half of the slate** (≥ 0.934 from Claude-4.5-Haiku onwards): when these judges flag a completion as hallucinated, they are rarely wrong, which is the operationally important property for a verifier sitting in front of a devel-

oper. The two smallest judges (GPT-4o-mini, GPT-4.1-mini) drop to 0.77–0.86, mirroring their lower Gold Acc. in Table 7. (2) **Recall is the bottleneck for every judge**. The spread on Recall (0.716–0.938) is $\sim 2\times$ the spread on Precision among the top six judges (0.934–0.986), so improvements in detection on DELULU will primarily come from catching the missed hallucinations rather than reducing false flags on golden completions. (3) The ranking by F1 and MCC closely tracks the ranking by both-correct accuracy in Table 7: Claude-4.5-Opus first on both, GPT-5.2-Codex second, Claude-4.6-Sonnet third, GPT-5.4 fourth, and the smaller judges trail in the same order. The two metric families therefore agree on judge ordering; we retain both-correct as the headline metric in the main text because it directly reflects the dual-completion structure of DELULU, but the F1/MCC summaries are robust to readers who weight false positives and false negatives differently.

D Curation Pipeline Details

Stage 1: Hallucination generation. From source files scraped from public GitHub repositories, CLAUDE SONNET generates paired golden and hallucinated completions using type-specific prompts (§F). Import hallucinations require structural rearrangement so the hallucinated import becomes the FIM completion target. We generate $\sim 10K$ samples per type per iteration. CLAUDE SONNET was selected as the sole generator after a pilot of four candidate frontier models (CLAUDE SONNET, GPT-5.1, GPT-5.2, GLM-4.7) showed it had the strongest prompt adherence—honoring the “modify exactly one element, keep everything else identical, do not signal that the output is hallucinated” constraint at the highest rate. Single-generator bias is mitigated downstream by the four-judge panel and the human-verification pass.

Stage 2: Multi-model judge evaluation. Four judges—GPT-5.1, GPT-5.2, GPT-5.2-CODEX, GLM-4.7—independently evaluate each sample. Each judge receives the FIM context paired with either the golden or hallucinated completion and produces a binary score with chain-of-thought reasoning, yielding $8N$ LLM calls per iteration, where N is the number of candidate samples evaluated in the iteration ($4 \text{ judges} \times 2 \text{ completions per sample}$).

Stage 3: Iterative adversarial sampling. We analyze judge disagreements (false positives and false negatives), extract reasoning embeddings using all-MiniLM-L6-v2, cluster them via K-means with silhouette-based k selection (typically $k \in [5, 10]$), and mine semantically similar code from the unlabeled corpus (cosine similarity in $[0.75, 0.99]$). The loop runs for three iterations. Several patterns emerge from the iteration-level results in Table 3: GLM-4.7 shows severe degradation on import ($0.17 \rightarrow 0.04 \rightarrow 0.03$), confirming that open-weights models are particularly vulnerable to ecosystem-specific hallucinations; GPT-5.2-CODEX maintains high accuracy on undefined variable (≥ 0.89), reflecting the relative simplicity of scope reasoning; method-hallucination accuracy is non-monotonic for some judges, suggesting that adversarial mining discovers qualitatively different failure modes in later iterations rather than simply scaling difficulty.

Stage 4: Difficulty-based selection. We score each sample

$$d(x) = w_h \sum_{j=1}^4 \mathbb{1}[\text{judge}_j(x_{\text{hall}}) = 1] + \quad (1)$$

$$w_g \sum_{j=1}^4 \mathbb{1}[\text{judge}_j(x_{\text{gold}}) = 0], \quad (2)$$

with $w_h = 1.0$ (false acceptance) and $w_g = 0.5$ (false rejection), requiring $d(x) \geq 1.0$. Language balancing targets $\lfloor N/7 \rfloor$ samples per language, prioritizing difficult samples.

The asymmetric weights reflect the asymmetric purpose of each judge call. A judge that *accepts* a hallucination ($w_h=1.0$) provides direct positive evidence that the hallucination is non-trivial: at least one frontier model with chain-of-thought reasoning was unable to flag the error, which is exactly the failure mode DELULU is designed to surface. A judge that *rejects* a golden completion ($w_g=0.5$) provides only indirect, label-confounded evidence (the rejection may track sample-quality noise such as unusual identifiers, project-specific idioms, or upstream-repo bugs rather than hallucination subtlety); we retain that signal but down-weight it by half. The threshold $d(x) \geq 1.0$ corresponds to “at least one judge fooled, or two judges over-rejecting the golden”, which on the Iter-0 pool retains the upper $\sim 58\%$ of samples by difficulty and removes the trivially-decidable lower tail. We confirmed empirically that thresholds in $\{1.0, 1.5, 2.0\}$ shift cell

sizes but do not change the cross-language ordering of difficulty, so we adopt the most permissive setting that still excludes universally-easy samples.

Distributed execution. The full pipeline runs on Databricks Asset Bundles with up to 16 concurrent judging tasks. Each iteration involves $\sim 80,000$ LLM calls ($4 \text{ judges} \times 2 \text{ completions} \times \sim 10\text{K}$ samples), with exponential-backoff retry (up to 5 attempts, 60 s max delay). Judge evaluation per iteration takes 7–30 hours depending on the model; clustering and mining add ~ 20 hours per iteration. Stage breakdown: Stage 1 (Generation) ~ 1 hour; Stage 2 (Judging) four parallel tasks of 7–30 hours; Stage 3 (Clustering) ~ 20 hours.

E Human Expert Review Protocol

After all candidate samples pass Docker-based execution verification (§2.3), they undergo a final human expert review. This step is performed *after* execution verification so that human effort is spent only on the small set of samples that have already been confirmed to compile and produce the expected errors.

Panel and workflow. Three expert annotators, each with professional software engineering experience across the seven target languages, review every Docker-verified sample using a dedicated annotation interface (Figure 5). The interface displays the prefix, suffix, and *both* completions *labelled* as golden and hallucinated; reviewers therefore validate that the labels are correct and that the pair is non-trivially distinguishable rather than guessing which is which. The annotator selects one of three actions:

- **Accept:** the tuple is valid, the golden completion is correct, and the hallucinated completion is a plausible but genuinely incorrect alternative.
- **Reject:** the tuple is fundamentally flawed (e.g., the golden completion is itself incorrect, or the hallucination is trivially detectable from a surface cue such as a leftover comment or formatting artifact).
- **Edit:** the hallucinated completion is modified by the annotator to make it more plausible or to fix a minor issue while preserving the hallucination type. Edited samples are automatically re-run through the full Docker execution pipeline to confirm that the revised hallucinated completion still triggers the expected runtime error class.

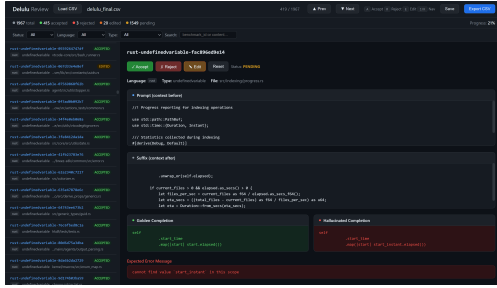


Figure 5: The expert annotation interface. Each sample displays the prefix, suffix, and the two completions labelled as golden and hallucinated. Annotators can accept, reject, or edit the hallucinated completion; edits trigger automatic re-verification via Docker.

Statistics. Of 1,957 Docker-verified candidate samples submitted for human review, 1,744 were accepted as-is, 6 were rejected (removed from the dataset), and 207 received edits to the hallucinated completion (some samples were edited and subsequently accepted, so the counts overlap). All 207 edited samples passed re-verification, confirming that the expert modifications preserved the intended runtime error. After removing the 6 rejected samples, the final released dataset contains 1,951 samples.

F Hallucination Generation Prompts

We use type-specific system prompts for CLAUDE SONNET. Each prompt instructs the model to modify exactly one aspect of the completion while preserving all other code. We show **method** and **import**; parameter and undefined variable follow the same structure.

System Prompt: Method Hallucination

You are given a code snippet divided into three parts:

- Prefix: the portion of code before the completion.
- Completion: the part generated by a code model. The completion always includes a method call.
- Suffix: the code that comes after the completion.

Your task: simulate a hallucination by modifying the completion:

- Replace the method name with a random, invented method name that does NOT appear in prefix/suffix.
- Keep everything else the same.
- The name should be plausible -- avoid naming it in a way that shows it is undefined or hallucinated.

Output exactly in this format:

Generated completion: <the hallucinated completion>

Do not modify prefix or suffix, add comments, or change formatting.

System Prompt: Import Hallucination

You are given a complete source code file. Read it carefully.

Your task: simulate a hallucination by REPLACING one real import statement with a hallucinated version:

- Pick one real import line from the file.
- Replace it with a plausible but FAKE import that does NOT exist (e.g., "from datautils import "DataProcessor", "import tensorflow").
- The fake module must NOT be a real package. It must cause an ImportError or ModuleNotFoundError.
- Do NOT add a new import. ONLY replace one line.

Output exactly in this format (two lines):

Original import: <the exact original line>
Hallucinated import: <the fake replacement line>

G Judge Evaluation Prompt

The same system prompt is used for all four judges during curation:

Judge System Prompt

You are a top-tier software engineer with deep expertise in software development and code review. Evaluate a candidate code completion that should connect the provided prefix and suffix inside the same file.

Score 1 ONLY when all of the following are true:

- The completion keeps the file syntactically valid.
- The completion is logically consistent with the surrounding context.
- The completion is relevant: it advances the task from the prefix toward the suffix.
- The completion does not hallucinate APIs, data, or behavior conflicting with the context.

Otherwise score 0.

Format your response exactly as:

Detailed Reasoning: <reasoning text>

Final Average Score for Completion: <0 or 1>

H Execution Verification: Full Details

Verification protocol. For each sample, we construct two complete source files by inserting the golden or hallucinated completion between the prefix and suffix. The golden file must compile/parse and run without errors. The hallucinated file must produce the specific error type corresponding to its hallucination category: `AttributeError` (method), `TypeError` (parameter), `NameError` (undefined variable), or `ImportError` (import). Samples failing either condition are excluded.

Docker infrastructure. We maintain seven per-language base images, each equipped with the full toolchain and standard package manager: Python (CPython 3.11 / pip), Java (OpenJDK 17 / Maven), TypeScript (Node.js 20 / npm), Go (1.21 / go modules), Rust (nightly / cargo), C++ (GCC 13 or Clang / apt), and C# (.NET 8.0 / NuGet). Each image includes a dependency-resolution module that analyzes the source file’s imports and installs the required packages before verification.

Standalone-file precheck. DELULU targets self-contained, file-level FIM evaluation, so verification is preceded by a static precheck that discards any source file whose dependency graph cannot be closed within the file itself: files that import sibling modules from the same repository, depend on project-internal build artefacts, or require harness code outside the file are rejected before any con-

tainer is built. The precheck retains roughly 5% of mined files, and this filter—rather than the verification stage that follows—is the dominant determinant of overall yield. The constraint is deliberate: it keeps every per-sample container lightweight and reproducible at the cost of excluding repository-level FIM scenarios, which are complementary and better served by repository-level benchmarks (see Appendix K).

CLAUDE SONNET-assisted fix loop. Real-world GitHub code frequently has external dependencies that are not immediately satisfiable in a clean container even after the precheck (system libraries, build flags, native toolchain configurations). Conditional on passing the precheck, roughly 0.4 of the assembled (golden, hallucinated) tuples verify successfully on the first attempt. For the remainder, we run a CLAUDE SONNET-assisted debugging loop: the error output is sent to CLAUDE SONNET, which proposes a fix (e.g., installing missing system libraries, adding package dependencies, or adjusting build configurations); the proposed fix is applied to the container environment and verification is retried. The loop runs up to three iterations per sample and raises the verification rate among precheck-passing files to ~ 0.75 . Figure 6 walks through one such case.

```
[23/186] cpp | method | cpp-method-5d87f2ec2546
doc/Programs/IMC2Electrons/vncdot.cpp

Attempt 1/3...
Golden: X (Import)
  * Asking Claude for fix...
  * Applying 1 fix(es)...
  $ apt-get install -y libarmadillo-dev
  X Fix commands failed
Attempt 2/3...
Golden: X (Import)
  * Asking Claude for fix...
  * Applying 2 fix(es)...
  $ apt-get update && apt-get install -y libarmadillo-dev libopenblas-dev liblapack-
  $ find / -name 'armadillo' 2>/dev/null | head -5
  / Fix applied - retrying...
Attempt 3/3...
Golden: X (none)
Halluc: X (Attribute)
  / Valid benchmark: goldenspass, halluc-fail
  * Committing image: hallubench-cpp-method-5d87f2ec2546.v1...
```

Figure 6: CLAUDE SONNET-assisted fix loop: a C++ sample requires three iterations to resolve missing dependencies before golden (pass) and hallucinated (expected fail) verifications both succeed.

Container finalization. Successfully verified samples are packaged as self-contained Docker containers with all code and dependencies baked in, then pushed to Azure Container Registry. Each container supports three invocation modes: `verify golden` (confirm golden compilation), `verify hallucinated` (confirm error production), and `verify patch` (evaluate a model-generated completion via stdin). This interface enables drop-in integration with any FIM model evaluation pipeline.

Docker Commands

```
## Pull a sample container
docker pull <registry>/<image_tag>

# Verify golden completion compiles
docker run <image> verify golden

# Verify hallucinated completion
# produces expected error
docker run <image> verify
hallucinated

# Evaluate a model-generated
# completion (via stdin)
echo "<completion>" | docker run -i <
image> verify patch

# Evaluate from a file (via volume
# mount)
docker run -v ./comp.txt:/input/patch
.txt <image> verify patch

# Retrieve sample metadata
docker run <image> info          # JSON
with all metadata
docker run <image> prompt       # FIM
prefix context
docker run <image> golden      #
correct completion text
docker run <image> hallucinated #
incorrect completion text
```

verify patch returns structured JSON: `{"is_valid": true|false, "error_message": "..."}.` Container images average 150–400 MB; containers run with `--network=none` (except C# which requires NuGet restore).

I Statistics: Full Details

Marginal language distribution and per-language complexity are reported in Tables 9 and 10.

Table 9: Language distribution.

Language	N	%
TypeScript	420	21.5
Python	374	19.2
Go	291	14.9
Rust	252	12.9
C#	246	12.6
Java	243	12.5
C++	125	6.4
Total	1,951	

Per-language complexity varies considerably within DELULU: C# samples average 255.3 LOC and 22.1 CC (enterprise-style code), while TypeScript averages 77.0 LOC and 9.6 CC (compact web-development patterns). This variation ensures the benchmark tests hallucination resilience across

diverse complexity profiles.

J Evaluation: Metrics, Per-Type, and Fine-Grained Results

Metric definitions. **pass@1** (execution-based): a completion *passes* if the assembled file compiles and runs without errors when the prediction is inserted. **Edit Similarity (ES)**: $1 - \text{Levenshtein}(\hat{c}, c_g) / \max(|\hat{c}|, |c_g|)$. **CodeBLEU** (Ren et al., 2020): equally-weighted combination of n-gram, weighted n-gram, AST, and dataflow matching, using language-specific parsers. **Hallucination Rate (HR)**: $\frac{1}{N} \sum_i \mathbb{I}[\text{sim}(\hat{c}_i, c_{h,i}) > \text{sim}(\hat{c}_i, c_{g,i})]$ with SequenceMatcher similarity.

pass@1 by hallucination type.

Code-quality metrics by language. Figure 7 reports CodeBLEU and Edit Similarity by language. Both correlate positively with pass@1 across the slate, but the correlation is not perfect: a model can produce a completion that is syntactically similar to the golden code yet functionally incorrect (or vice versa)—motivating our multi-metric approach. Note that Qwen-0.5B’s Edit Similarity is unusually high relative to its pass@1 because the line-budget cap (§4) clips the most degenerate suffixes for that model only, leaving short well-formed prefixes that score generously on character-level similarity even when the underlying API knowledge is wrong.

Per-language and per-(language × type) heatmaps. Figure 8 shows pass@1 heatmaps per model with language (rows) × hallucination type (columns), computed on the 950 unique FIM contexts. Several patterns emerge: (i) **Import across all languages** is consistently the weakest column at every scale, with C# and Go imports proving especially challenging—Qwen-32B reaches only 0.76 on C#×Import and 0.68 on Go×Import; (ii) **C++ method/parameter cells** remain unstable across scales (the small 32 unique-context C++ cell is sensitive to a handful of failures, e.g. 0.75 on Method at both 14B and 32B); (iii) **0.5B is uniformly poor** across all cells; (iv) improvement with scale concentrates on specific (language, type) pairs rather than improving uniformly, and the 3B>7B inversion (§4) is visible cell-by-cell—several cells regress between 3B and 7B (e.g. Python×Param., C#×Param., Go×Method) before recovering at 14B/32B.

Table 10: Per-language complexity metrics for DELULU.

Language	N	Pfx LOC	Cmp LOC	Tot LOC	Pfx Tok.	Cmp Tok.	CC
C++	125	42.5	1.0	83.5	319.2	7.5	11.3
C#	246	110.2	2.1	255.3	1118.5	20.7	22.1
Go	291	97.6	1.5	210.4	945.7	12.9	24.9
Java	243	59.1	1.1	123.8	426.2	10.7	16.0
Python	374	42.9	1.7	152.1	344.8	11.8	13.8
Rust	252	65.0	1.6	174.5	540.3	13.1	19.2
TypeScript	420	30.8	3.7	77.0	203.4	25.8	9.6
Overall	1,951	61.7	2.0	152.6	534.4	15.8	16.4

Table 11: pass@1 by hallucination type, Qwen2.5-Coder slate, computed on the 950 unique FIM contexts. Import is consistently the most challenging across all model sizes.

Type	0.5B	1.5B	3B	7B	14B	32B
Import	.378	.540	.653	.630	.736	.771
Method	.474	.695	.766	.747	.831	.851
Parameter	.428	.723	.855	.766	.906	.906
Undef. Var.	.491	.709	.858	.759	.851	.900

K Discussions

K.1 Benchmark difficulty and validity.

Four pieces of evidence support the claim that DELULU is genuinely challenging rather than a pattern-matching test. (1) The adversarial mining loop is empirically effective: cross-judge both-correct accuracy on the unverified Iter-0 pool decreases monotonically across iterations (Table 3), confirming that progressively harder samples are surfaced. (2) On the final execution-verified release, even the strongest detection judge (Claude-4.5-Opus) achieves 92.1% both-correct accuracy (Table 7), with import hallucinations remaining the hardest category at 83.4%; detection is therefore unsolved on DELULU even for frontier judges. (3) Docker-based execution verification guarantees that every retained hallucination produces the categorical runtime error class, removing the LLM-label noise that inflated the Iter-0 numbers. (4) The strongest of the 11 generators we evaluate (Qwen2.5-Coder-32B-Instruct) reaches 84.5% pass@1 (on the 950 unique FIM contexts) and no cross-family model exceeds 0.77 Edit Similarity, so neither the judge frontier nor the FIM-generator frontier solves the benchmark. The two-stage human-review pass (Appendix E; 1,957 Docker-verified candidates $\rightarrow$ 1,951 released, with 207 edits re-verified through Docker) further removes residual single-generator artefacts that could otherwise be confused with intrinsic difficulty.

K.2 Scaling behavior and capability emergence.

The transition from 0.5B (43.6%) to 1.5B (64.7%) suggests a *capability threshold*: below 1.5B parameters, models lack the minimum semantic understanding needed for reliable FIM completion in hallucination-prone contexts—and even after the line-budget cap, the 0.5B model only matches the gold completion byte-for-byte on 22.8% of samples and reaches an Edit Similarity of 0.625, well below every other Qwen size. The 3B>7B inversion (76.7% vs. 71.1%, with 7B showing *lower* Edit Similarity (0.625 vs. 0.693) and Exact Match (0.402 vs. 0.440) than 3B) is reproducible across all four metrics in our single-seed run; we report it as an open anomaly and decline to attribute it to a specific cause without multi-seed verification, layer-wise probing, or training-recipe details that are not publicly documented (§4). The modest gap between 14B (81.5%) and 32B (84.5%) suggests that reducing hallucination susceptibility beyond a certain scale may require targeted interventions—training-data curation, post-training alignment on hallucination-specific signals, or specialized fine-tuning—rather than simply increasing model size.

K.3 Hallucination type difficulty hierarchy.

Across the Qwen scaling slate (Table 11) and the cross-family slate, *import is consistently the hardest category*: at 32B, pass@1 on Import is 0.771 versus 0.851–0.906 on the other three categories, and the same ordering holds at every smaller scale.

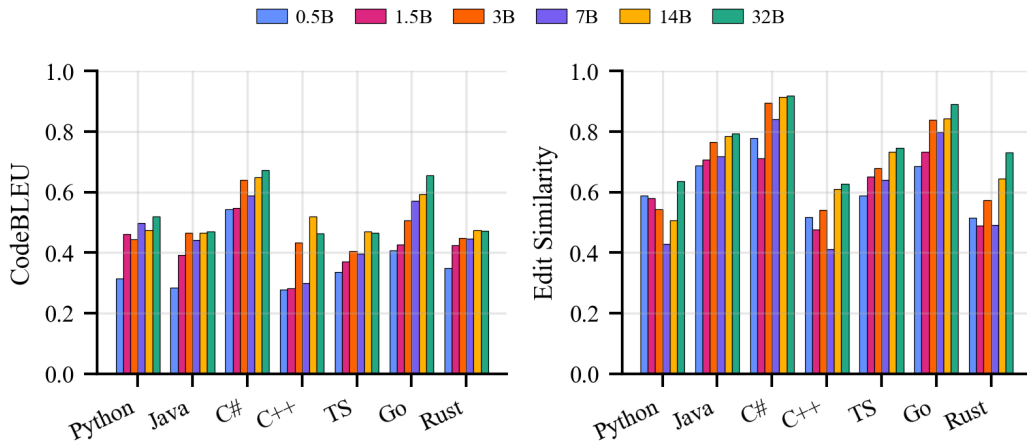


Figure 7: CodeBLEU (left) and Edit Similarity (right) by language across Qwen2.5-Coder model sizes. Larger Qwen sizes generally improve both metrics; Qwen-0.5B’s Edit Similarity is inflated by the line-budget cap and should be compared against pass@1 rather than read as an indicator of completion quality.

The remaining three categories (Method, Parameter, Undef. Var.) cluster within a few percentage points of each other at 32B and we therefore describe them as statistically indistinguishable on the present sample sizes. The Import gap is plausibly explained by the semantic demand of each category: Import requires memorized knowledge of the actual package ecosystem (knowing that `sklearn.neural` does not exist demands traversing the ecosystem hierarchy), whereas Method, Parameter, and Undef. Var. admit at least partial inference from local context—surrounding identifiers, type annotations, or scope. Per-cell confidence intervals would sharpen the within-tier comparisons.

K.4 Dual use: generation and detection.

DELULU serves two purposes: (i) *generation evaluation*—running model inference on DELULU and measuring pass@1 quantifies how often a FIM model produces hallucinated completions; (ii) *detection evaluation*—the judge framework from our curation pipeline can be repurposed to evaluate how well models detect hallucinations in suggested code, increasingly important for code-review assistants and agentic coding systems. The paired golden/hallucinated completions, together with difficulty scores, also provide a potential training signal for supervised or preference-based fine-tuning.

L Qualitative Examples

We present one representative sample per hallucination type. Each example shows the FIM context, the golden (correct) completion, and the hallucinated (incorrect) completion with its resulting error.

Import Hallucination.

Context (prefix tail)

```
import os, sys
from pathlib import Path
```

✓ Golden Completion

```
from PIL import Image, ImageFilter
```

✗ Hallucinated Completion

```
from Pillow import Image, ImageFilter
```

`ModuleNotFoundError: No module named 'Pillow'`

Method Hallucination.

Context (prefix tail)

```
@app.route('/fact/<fact>',
            defaults={'env': ..., 'value': None
                    })
@app.
```

✓ Golden Completion

```
route('/<env>/fact/<fact>', defaults={'value': None})
```

✗ Hallucinated Completion

```
endpoint('/<env>/fact/<fact>', defaults={'value': None})
```

`endpoint() got an unexpected keyword argument 'defaults'`

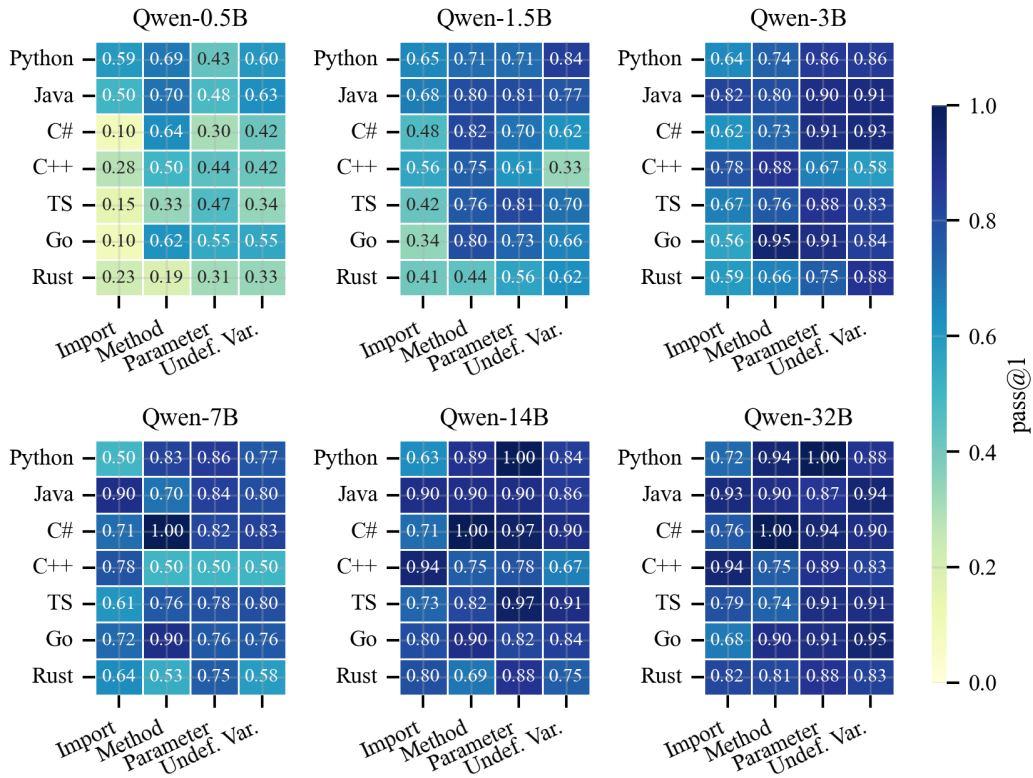


Figure 8: pass@1 heatmaps per model: language (rows) × hallucination type (columns).

Parameter Hallucination.

Context (prefix tail)

```
sa.column('name', sa.String),
sa.column('abbr', sa.String),
sa.
```

✓ Golden Completion

```
column('service', sa.String)
```

✗ Hallucinated Completion

```
column('service', sa.String, nullable=False)
```

column() got an unexpected keyword argument 'nullable'

Undefined Variable Hallucination.

Context (prefix tail)

```
class JudgeDecisionSchema(BaseModel):
    """Schema for judge decisions"""
    approved: bool =
```

✓ Golden Completion

```
Field(..., description="Whether the card is approved")
```

✗ Hallucinated Completion

```
Field(..., description=approval_default)
```

NameError: name 'approval_default' is not defined

M Source Data Provenance

The benchmark draws from a corpus of ~1.1 million code completion examples mined in October 2025 from ~11,975 public GitHub repositories across 9 languages and 464 third-party APIs. For each language, ~25 target packages were selected by a consensus of three signals: (i) adoption frequency in anonymized code completion telemetry, (ii) LLM-based ecosystem ranking, and (iii) curated priority lists from language-specific engineering teams. Each target package was pinned to a minimum version released within a 4-month window of the mining date; only repositories declaring a dependency at or above that version were retained. C++ libraries were selected via expert curation and

matched by include-path patterns, as C++ lacks a standardized package manager. Source files were filtered to the 2nd–98th percentiles for file size and line count. API call sites were extracted using tree-sitter AST parsing, classified by GPT-4o-mini, and split into fill-in-the-middle (prefix, completion, suffix) triplets at the call site’s byte offsets.

From this corpus, 1,951 examples were sampled across 7 languages and 319 repositories. All examples are drawn exclusively from public GitHub repositories. We traced each example to its source repository and retrieved metadata. Across the 319 repositories, the median GitHub star count is 81, with a range of 3 to 76,366 (mean: 1,890); 56% have ≥ 50 stars and 22% have $\geq 1,000$ stars. Each released sample includes the source repository URL and detected license metadata for provenance purposes.

For languages with standardized package managers, the source corpus retains only repositories that declared dependencies on package versions released within 4 months of the October 2025 mining date. Table 12 reports the earliest target package release date per language.

Table 12: Earliest target package release date per language at the time of mining (October 2025).

Language	Earliest Package Date	Age at Mining
C#	April 2025	~6 months
Java	May 2025	~5 months
PHP	June 2025	~4 months
Python	June 2025	~4 months
Rust	June 2025	~4 months
Go	July 2025	~3 months
TypeScript	July 2025	~3 months
JavaScript	July 2025	~3 months
C++	N/A (curated list)	—

N Acknowledgment of LLM Assistance

We acknowledge the use of large language model (LLM) tools to assist with limited aspects of this work, including language editing and polishing of text, generation of visualization code and plots, and debugging support for experimental pipelines. All scientific contributions, including the problem formulation, benchmark design, methodology, analysis, and final written content, are the original work of the authors. The authors carefully reviewed and verified all outputs from LLM tools to ensure accu-

racy and correctness.